Distributed Real Time Control Systems: Stage 2

André Salvaterra(106175)*, Marco Matos* (105932), Tomás Modesto* (105944)
Instituto Superior Técnico, Lisbon, Portugal
*andre.salvaterra@tecnico.ulisboa.pt, †marco.matos@tecnico.ulisboa.pt, ‡tomas.modesto@tecnico.ulisboa.pt

I. Introduction

This stage addresses the implementation of a distributed cooperative control system for a network of $N = 3$ luminaires interconnected through a CAN-BUS network. The objective is to minimize global energy consumption while ensuring user comfort, defined as maintaining illuminance above prescribed reference levels and avoiding perceptible flicker.

Unlike centralized approaches, the proposed solution is fully distributed: each node operates as an autonomous decision-making unit and cooperates with others through asynchronous communication. This architecture improves scalability and robustness, as the system remains operational even in the presence of node or communication failures. Furthermore, the system is designed to operate with a variable number of nodes, allowing flexible deployment.

II. System Architecture and Real-Time Implementation

The system leverages the dual-core architecture of the RP2040 microcontroller together with FreeRTOS to ensure deterministic real-time execution. Core 0 runs the FreeRTOS scheduler and handles all time-critical tasks, including the main 100 Hz control loop (`controllerTask_dd`), the 5 Hz distributed optimization, the calibration state machine (`calibrationTask`), and the serial interface. Periodic execution is achieved using `vTaskDelayUntil`, while inter-task synchronization and data exchange rely on FreeRTOS primitives.

Core 1 is dedicated exclusively to CAN bus management, executing the MCP2515 driver via SPI in an isolated loop.

Inter-core communication is performed through thread-safe FreeRTOS queues (`canTxQueue` and `canRxQueue`) and task notifications for the calibration FSM (Finite State Machine). FreeRTOS proved to be an excellent choice for this project, providing a priority-based preemptive scheduler and efficient synchronization primitives that enable clean separation of concerns and robust real-time behaviour with minimal overhead.

This architecture offers excellent scalability, as increases in network traffic or the number of nodes do not degrade the timing determinism of the local control loops.

III. Communication Infrastructure and Hub Function

A lightweight communication protocol was developed on top of the CAN bus, using fixed 8-byte frames that carry a source ID, destination ID, command byte, sub-command byte, and floating-point value. This format ensures high numerical precision for optimization variables while providing deterministic and bounded transmission latency.

To reduce network load, an event-triggered strategy is employed: nodes only transmit when new optimization data or commands are generated. In addition, the MCP2515 hardware acceptance filters are used to discard irrelevant messages at the hardware level, significantly lowering CPU overhead on each node.

Any node connected to a PC via the serial interface can act as a communication hub. It forwards measurements from the distributed network to the PC and routes commands to specific nodes, enabling centralized monitoring and configuration without compromising the fully distributed nature of the control algorithm.

IV. Network Initialization and System Identification

A distributed boot procedure allows nodes to dynamically discover each other regardless of startup order. Each node identifies active peers and initializes the required data structures, enabling operation with an arbitrary number of luminaires.

System calibration is performed using a token-ring finite state machine. Each node sequentially activates its LED at full intensity while the remaining nodes measure the resulting illuminance. This procedure allows each node to estimate its corresponding row of the coupling matrix K, capturing both direct and cross-coupling effects.

The system continuously estimates the ambient light component d by subtracting the predicted contribution of all luminaires from the measured illuminance, allowing adaptation to external disturbances such as daylight variations.

V. Control Architecture

Each node implements a local PID controller to regulate illuminance. Anti-windup is enforced by clamping the integral state when the actuator reaches its limits (0–100% duty cycle), preventing integrator saturation during transients. Sensor measurements are pre-processed with an 11-sample median filter to suppress noise and outliers.

At the global level, the distributed control objective is formulated as:

$$\min_{\mathbf{u}} \ \mathbf{c}^\top \mathbf{u} \quad \text{s.t.} \quad K\mathbf{u} + \mathbf{d} \geq \mathbf{L}, \quad \mathbf{0} \leq \mathbf{u} \leq \mathbf{100}$$

where c_i is the energy cost of luminaire i, $K \in R^{N \times N}$ is the photometric coupling matrix, $\mathbf{d}$ is the background illuminance vector, and $\mathbf{L}$ is the vector of illuminance lower bounds.

VI. Dual Decomposition

As a first approach to distributed optimisation, a dual decomposition method was implemented. The global problem is relaxed by dualising the illuminance coupling constraint, associating a Lagrange multiplier λ_i (price) with each node. Each node i solves a decoupled local subproblem:

$$u_i \leftarrow \frac{-c_i + \lambda_i k_{ii} + \sum_{j \neq i} \lambda_j k_{ji}}{2q}$$

where k_{ii} is the self-gain, k_{ji} is the effect of node i's LED on node j's sensor, and q is a quadratic regularisation weight. The dual variable is updated using subgradient ascent on the constraint violation:

$$\lambda_i \leftarrow \max(0,\ \lambda_i + \alpha(L_i - y_i))$$

where y_i is the measured illuminance and α is the step size. Nodes exchange prices $\lambda_j k_{ji}$ over CAN-BUS after each iteration.

In practice, this formulation exhibited slow convergence and sensitivity to the choice of α, consistent with the known limitations of subgradient methods. A low-pass filter on the primal update ($u \leftarrow 0.5\,u + 0.5\,u_{\text{prev}}$) was required to damp oscillations. These limitations motivated the adoption of ADMM, which replaces the subgradient step with an augmented Lagrangian that provides faster and more robust convergence.

VII. Distributed Optimisation via ADMM

The global problem is coupled through K, making a centralised solution incompatible with the distributed architecture. The Alternating Direction Method of Multipliers (ADMM) is adopted to decompose the problem into per-node subproblems coordinated through a consensus variable.

Each node i maintains a local estimate $\mathbf{u}_i$ of the full duty cycle vector and iterates over three steps per consensus round. In the *primal update*, node i minimises its augmented local cost:

$$\mathbf{u}_i \leftarrow \arg\min_{\mathbf{u}}\ \mathbf{c}^\top \mathbf{u} + \boldsymbol{\lambda}_i^\top (\mathbf{u} - \bar{\mathbf{u}}) + \frac{\rho}{2}\|\mathbf{u} - \bar{\mathbf{u}}\|^2$$

Since no closed-form solution exists under the illuminance constraint, this subproblem is solved via Projected Gradient Descent (PGD) with 50 inner iterations. Each PGD step descends the augmented cost gradient, projects onto the illuminance feasibility halfspace via a correction proportional to the K-row gains, and finally applies a box projection to enforce $\mathbf{u} \in [0, 100]$.

In the *consensus update*, each node broadcasts its estimate over CAN-BUS and computes the network average $\bar{\mathbf{u}} = \frac{1}{N}\sum_i \mathbf{u}_i$. In the *dual update*, the multiplier is incremented as $\boldsymbol{\lambda}_i \leftarrow \boldsymbol{\lambda}_i + \rho(\mathbf{u}_i - \bar{\mathbf{u}})$, with symmetric clamping to $[-5,\ 5]$ to prevent dual variable windup.

The background illuminance d is estimated online at each iteration as:

$$d_k = \max\bigl(0,\ y_k - \mathbf{k}_i^\top \mathbf{u}_k\bigr)$$

where y_k is the current LDR measurement and $\mathbf{k}_i$ is the i-th row of K. This makes the optimiser robust to slow ambient light variations without requiring a separate disturbance estimator. A hardware failsafe forces all nodes to 100% duty cycle if the illuminance constraint remains violated by more than 2 lux after the PGD solver converges, ensuring safety under extreme conditions.

VIII. Hierarchical Integration and Performance

The system follows a two-layer hierarchical structure: the upper ADMM layer runs at 5 Hz computing optimal duty cycle references, while the lower PID layer tracks them at 100 Hz. Compared to a non-cooperative baseline, the distributed approach reduces energy consumption by exploiting photometric coupling between luminaires. Fault tolerance is preserved since no central coordinator exists.

IX. Conclusion

The combination of dual-core FreeRTOS scheduling, fixed-frame CAN-BUS, token-ring calibration, and ADMM-based optimisation yields a scalable and fault-tolerant architecture. The embedded PGD solver makes the algorithm tractable on microcontrollers without matrix inversion, at the cost of approximate subproblem solutions.

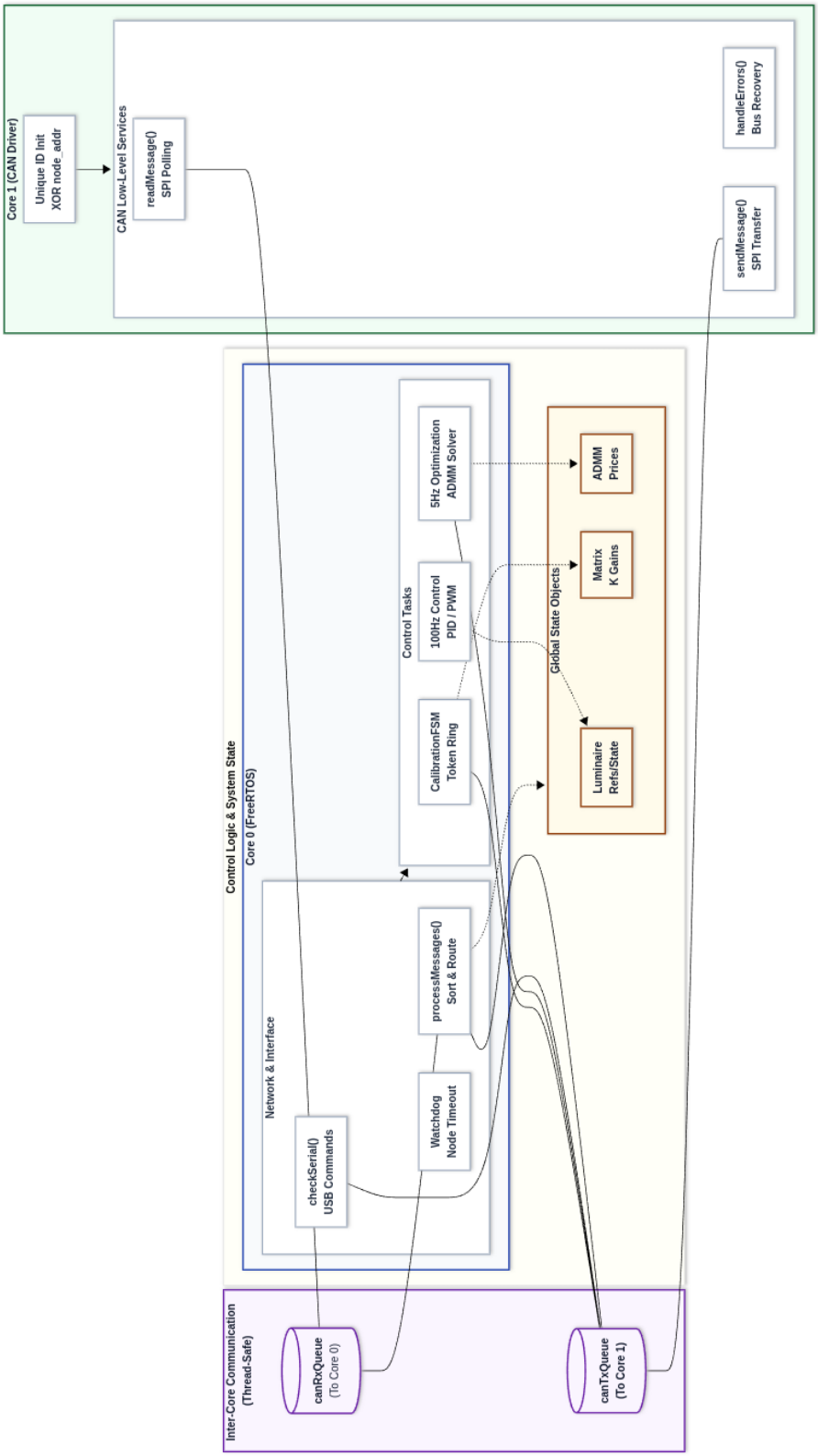


Fig. 1: RP2040 dual-core architecture. Core 1 manages the low-level CAN/SPI driver, while Core 0 executes FreeRTOS control loops (ADMM/PID) and application logic. Inter-core synchronization is maintained via non-blocking queues to ensure real-time determinism.

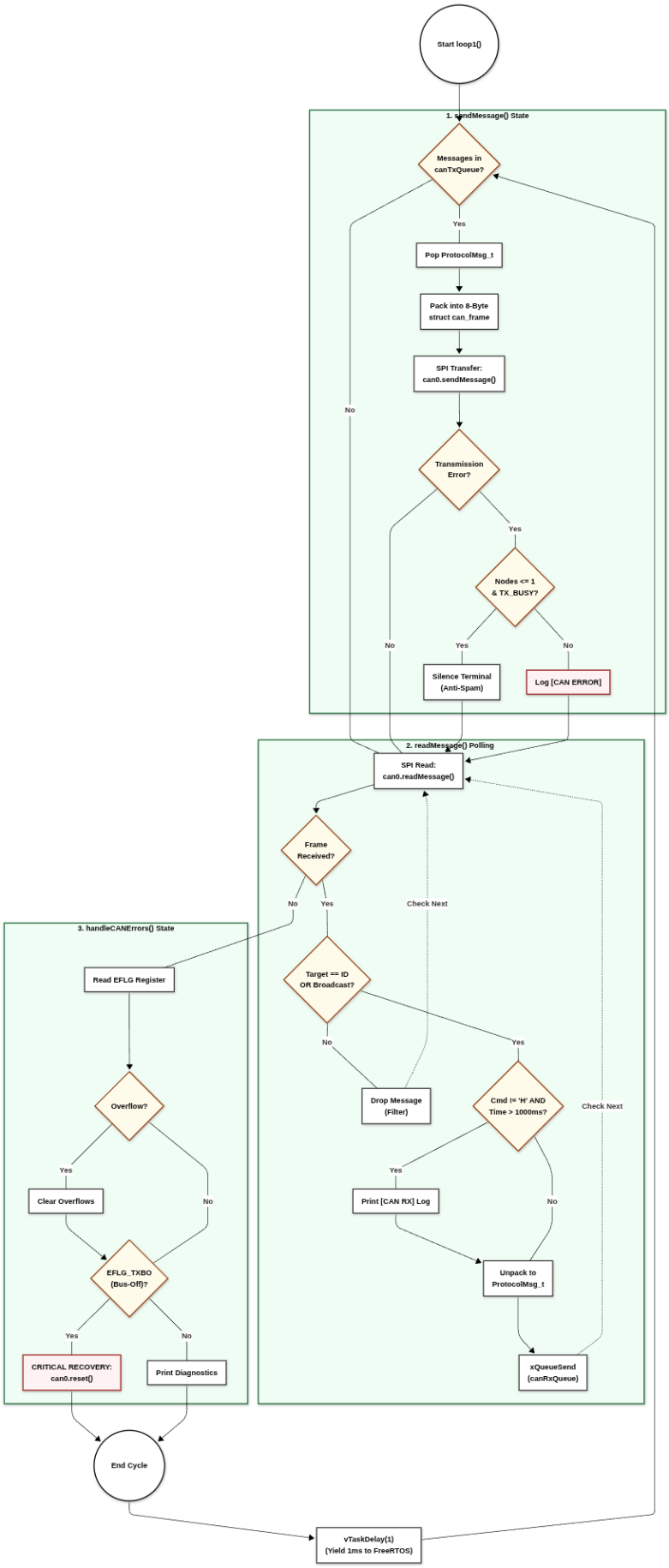


Fig. 2: Execution flow of the bare-metal CAN driver on Core 1. The routine asynchronously manages: (1) Transmission, draining the inter-core queue via SPI; (2) Reception, filtering frames by target ID; and (3) Error Handling, interrogating the EFLG register for autonomous buffer recovery and Bus-Off resets. The cycle ensures deterministic I/O without stalling the application layer.

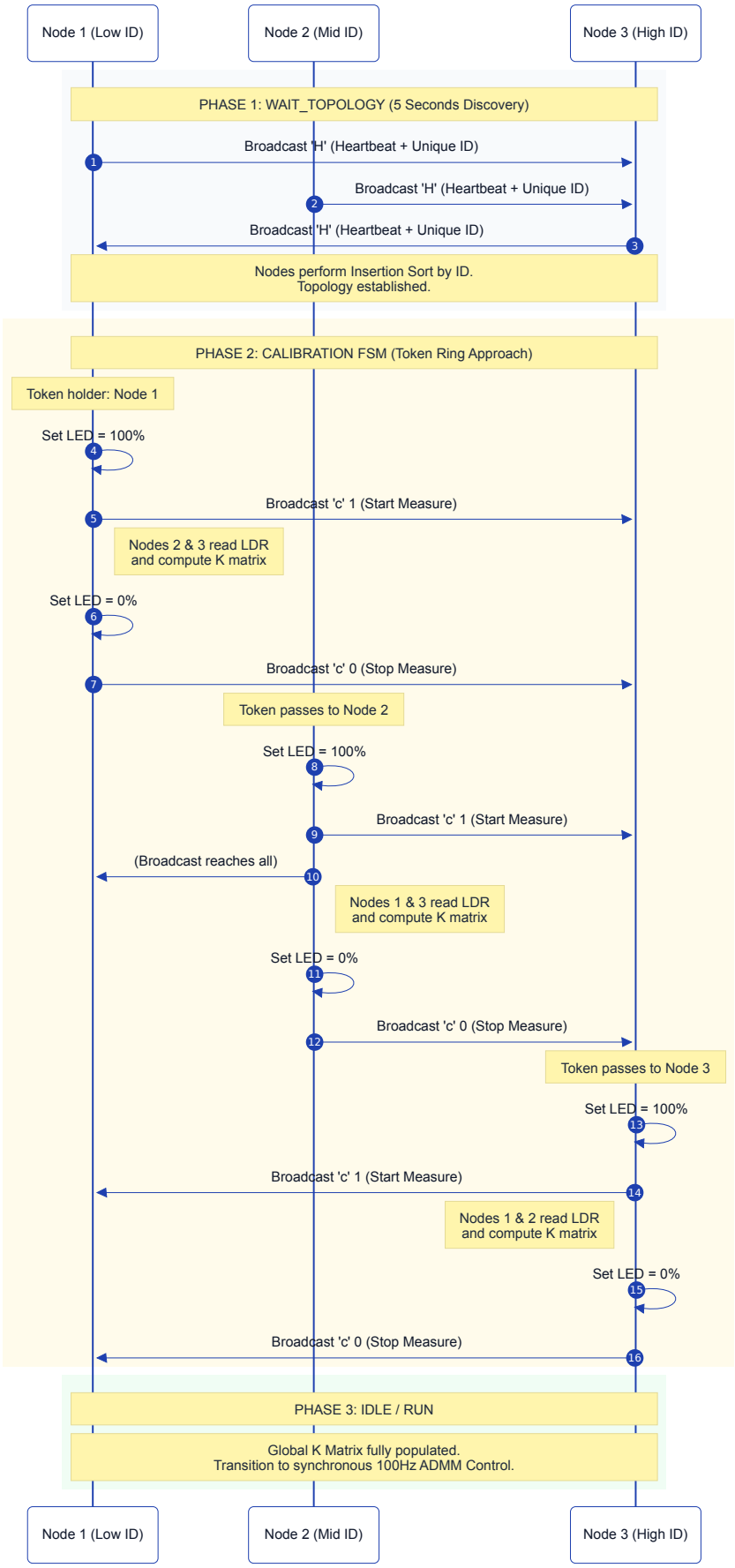


Fig. 3: Sequence of the distributed calibration protocol. Phase 1 (Discovery): nodes broadcast IDs to establish a sorted topology consensus via insertion sort. Phase 2 (Estimation): a token-ring FSM manages sequential 100% duty cycle pulses to populate the cross-coupling K matrix without collisions. Phase 3 (Execution): the system transitions into the synchronous 100Hz ADMM/PID control loop.

CAN Header	Data Payload (8 Bytes) - `ProtocolMsg_t`				
Standard CAN ID	**Sender ID**	**Target ID**	**Command**	**Sub-command**	**Value (Float)**
11 bits	1 Byte	1 Byte	1 Byte	1 Byte	4 Bytes

Fig. 4: **Custom CAN-bus frame structure implemented on Core 1.** The standard 11-bit CAN identifier is utilized for hardware-level collision avoidance. The 8-byte payload is fully optimized to encapsulate the `ProtocolMsg_t` structure, allocating 1 byte each for routing (Source/Destination IDs) and execution logic (Command/Subcommand), while dedicating the remaining 4 bytes to transmit a 32-bit floating-point optimization value without precision loss.

Protocol Design and Payload Optimization

The design of the CAN-bus communication protocol was strictly tailored to maximize bus utilization while guaranteeing deterministic latency for the real-time distributed control loops. A Standard Frame format with an 11-bit identifier was selected over the Extended 29-bit format. Given the confined nature of the local luminaire network, 11 bits provide more than sufficient address space while significantly reducing the frame overhead, thereby improving bandwidth efficiency and minimizing transmission times. Furthermore, the 11-bit ID natively facilitates hardware-level arbitration (CSMA/CD+AMP), naturally prioritizing critical network messages based on node addresses.

At the application layer, the 8-byte data payload was optimized to perfectly encapsulate the system's core data structure (`ProtocolMsg_t`) without requiring padding bits. As detailed in Figure 4, the payload is functionally partitioned:

- **Routing (2 Bytes):** The first two bytes explicitly define the *Source ID* and *Destination ID*. This enables efficient hardware and software filtering, isolating unicast traffic from network-wide broadcasts (e.g., target ID 255).
- **Execution Logic (2 Bytes):** The *Command* and *Sub-command* bytes provide a wide operational vocabulary (up to 2^{16} unique instructions), allowing the parser to decode operations ranging from network discovery to local state overrides.
- **Optimization Data (4 Bytes):** The remaining 4 bytes are dedicated to transmitting a 32-bit floating-point value. Sending the raw float is a critical design choice for the ADMM distributed algorithm. It completely eliminates the quantization errors and precision loss that would occur if the dual variables (prices) and coupling gains were compressed into smaller data types, thereby ensuring the mathematical convergence and stability of the optimization solver.

X. Real-Time Performance and Schedulability Analysis

To formally validate the deterministic behavior of the proposed dual-core architecture, execution time profiling was conducted on the physical RP2040 hardware. Rather than relying on standard RTOS tick counts—which suffer from millisecond-level quantization errors—measurements were captured using the native 64-bit hardware timer (`time_us_32()`), providing non-blocking microsecond resolution.

Table I summarizes the Average Execution Time and the Worst-Case Execution Time (WCET) for the three critical tasks during steady-state cooperative control.

TABLE I: Task Execution Time Profiling (Hardware Timer)

Task Name	Core	Avg Time (μs)	WCET (μs)
PID Control Loop (100Hz)	0	147.20	327
ADMM PGD Solver (8Hz)	0	878.44	1326
CAN Bare-Metal Driver	1	21.95	512

Note: Profiling data acquired over continuous operation. The 100 Hz PID loop operates under a strict $10000\,\mu s$ deadline. The recorded WCET of $327\,\mu s$ guarantees execution with over 96% computational headroom.

A. Statistical Evaluation of Determinism

The empirical data solidifies the viability of the RP2040 for complex distributed control:

1) **High-Frequency Control Headroom:** The primary PID control loop operates at 100 Hz, dictating a strict relative deadline of $10,000\,\mu s$. The profiler reveals that the PID task, including sensor filtering and anti-windup logic, requires a WCET of only $327\,\mu s$. This means the critical control path consumes merely 3.27% of the available CPU time, leaving immense headroom for FreeRTOS context switching and preventing any deadline misses.
2) **Mathematical Solver Efficiency:** The ADMM solver, which executes 50 internal iterations of Projected Gradient Descent (PGD) per cycle, demands an average of $878.44\,\mu s$ and peaks at $1326\,\mu s$. Because this task is scheduled at a slower rate ($\sim$8 Hz, or $120\,ms$ periods), its computational burden is negligible. Even in its absolute worst-case scenario, it executes fast enough to theoretically run inside the 100 Hz loop without breaking the deadline.
3) **Asynchronous I/O Isolation:** The bare-metal CAN driver on Core 1 demonstrates an exceptional average execution time of just $21.95\,\mu s$. The observed WCET of $512\,\mu s$ correlates with periods of dense network bursts (e.g., ADMM vector broadcast phases). Because this execution is strictly isolated on Core 1 and communicates with Core 0 via lock-free queues, these network latency spikes are entirely invisible to the PID loop, completely eradicating network-induced jitter.

Appendix A
Mathematical Formulation and Algorithm Implementation

This appendix details the mathematical formulation of the distributed optimization algorithms evaluated in this project: Dual Decomposition and the Alternating Direction Method of Multipliers (ADMM). The objective is to minimize global energy consumption while ensuring local illuminance constraints are met.

The general optimization problem is formulated as:

$$\text{minimize} \quad J(\mathbf{u}) = \sum_{i=1}^{N} \left(c_i u_i + q_i u_i^2\right) \tag{1}$$

$$\text{subject to} \quad \sum_{j=1}^{N} k_{ij} u_j + d_i \geq L_i, \quad \forall i \in \{1, \ldots, N\} \tag{2}$$

$$0 \leq u_i \leq 100, \quad \forall i \in \{1, \ldots, N\} \tag{3}$$

where u_i is the duty cycle (control effort) of luminaire i, c_i is the linear energy cost, q_i is a small quadratic regularization term, k_{ij} represents the optical cross-coupling gain from node j to sensor i, d_i is the background illuminance, and L_i is the target lower bound illuminance for node i.

A. Dual Decomposition

Dual Decomposition separates the global optimization problem into local sub-problems by relaxing the coupling constraints using the Lagrangian.

The Lagrangian is defined as:

$$L(\mathbf{u}, \boldsymbol{\lambda}) = \sum_{i=1}^{N} \left(c_i u_i + q_i u_i^2\right) + \sum_{i=1}^{N} \lambda_i \left(L_i - d_i - \sum_{j=1}^{N} k_{ij} u_j\right) \tag{4}$$

where $\lambda_i \geq 0$ is the dual variable (shadow price) associated with the illuminance constraint of node i.

The problem decomposes into local optimization problems for each node i:

$$\min_{u_i \in [0,100]} \left(c_i u_i + q_i u_i^2 - u_i \sum_{j=1}^{N} \lambda_j k_{ji}\right) \tag{5}$$

Note that $\sum_{j=1}^{N} \lambda_j k_{ji}$ represents the sum of prices from all nodes j that are influenced by node i's light.

The algorithm iteratively updates the primal variables (u_i) and dual variables (λ_i) using a gradient ascent method:

Algorithm 1 Dual Decomposition Update at Node i (Iteration k)

Receive prices $p_{ji}^{(k)} = \lambda_j^{(k)} k_{ji}$ from neighbors j
Primal Update: Minimize local Lagrangian
$u_{raw} \leftarrow \frac{-c_i + \lambda_i^{(k)} k_{ii} + \sum_{j \neq i} p_{ji}^{(k)}}{2q_i}$
$u_{raw} \leftarrow \text{clamp}(u_{raw}, 0, 100)$
$u_i^{(k+1)} \leftarrow 0.5 \cdot u_{raw} + 0.5 \cdot u_i^{(k)}$ {Low-pass filtering for stability}
Broadcast primal variable $u_i^{(k+1)}$ to neighbors
Receive primal variables $u_j^{(k+1)}$ from neighbors
Dual Update: Gradient ascent on constraint violation
$\lambda_i^{(k+1)} \leftarrow \lambda_i^{(k)} + \alpha \left(L_i - d_i - \sum_{j=1}^{N} k_{ij} u_j^{(k+1)}\right)$
$\lambda_i^{(k+1)} \leftarrow \max(0, \lambda_i^{(k+1)})$
Broadcast computed prices $p_{ij}^{(k+1)} = \lambda_i^{(k+1)} k_{ij}$ to neighbors j

B. Alternating Direction Method of Multipliers (ADMM)

ADMM enhances the robustness of standard dual decomposition by introducing a quadratic penalty term via the Augmented Lagrangian, which improves convergence characteristics even under less strict convexity assumptions. However, this quadratic penalty couples the variables, preventing straightforward decomposition.

To enable distributed computation, consensus ADMM reformulates the problem by introducing local copies of the global variable vector, $\mathbf{x}_i$, and enforcing consensus constraint $\mathbf{x}_i = \mathbf{z}$, where $\mathbf{z}$ is the global consensus variable.

The Augmented Lagrangian for the consensus problem is:

$$L_\rho(\mathbf{x}_i, \mathbf{z}, \boldsymbol{\lambda}_i) = \sum_{i=1}^{N} \left(f_i(\mathbf{x}_i) + \boldsymbol{\lambda}_i^T(\mathbf{x}_i - \mathbf{z}) + \frac{\rho}{2}\|\mathbf{x}_i - \mathbf{z}\|_2^2 \right) \tag{6}$$

where $\rho > 0$ is the penalty parameter and $f_i(\mathbf{x}_i)$ encapsulates the local cost function and local constraints (evaluating to $+\infty$ if constraints are violated).

The iterative updates at iteration k simplify to:

$$\mathbf{x}_i^{(k+1)} := \arg\min_{\mathbf{x}_i \in \mathcal{C}_i} \left(f_i(\mathbf{x}_i) + \boldsymbol{\lambda}_i^{(k)T}(\mathbf{x}_i - \boldsymbol{\mu}^{(k)}) + \frac{\rho}{2}\|\mathbf{x}_i - \boldsymbol{\mu}^{(k)}\|_2^2 \right) \tag{7}$$

$$\boldsymbol{\lambda}_i^{(k+1)} := \boldsymbol{\lambda}_i^{(k)} + \rho(\mathbf{x}_i^{(k+1)} - \boldsymbol{\mu}^{(k+1)}) \tag{8}$$

where $\boldsymbol{\mu}^{(k)} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i^{(k)}$ is the network average.

The core challenge lies in the primal update ($\mathbf{x}_i$-minimization), which involves solving a constrained quadratic program. To achieve this on computationally constrained microcontrollers without relying on external solvers, we implemented a Projected Gradient Descent (PGD) approach.

Algorithm 2 Consensus ADMM Update at Node i (Iteration k)

Receive local estimates $\mathbf{x}_j^{(k)}$ from all neighbors j
Global Average: Compute $\boldsymbol{\mu}^{(k)} \leftarrow \frac{1}{N}\sum_{j=1}^{N} \mathbf{x}_j^{(k)}$
Primal Update via PGD: Find optimal local state vector $\mathbf{x}_i^{(k+1)}$
Initialize inner estimate $\mathbf{v} \leftarrow \mathbf{x}_i^{(k)}$
for $m = 1$ to 50 **do**
  // 1. Gradient Step towards consensus
  $\mathbf{v}_{grad} \leftarrow \boldsymbol{\mu}^{(k)} - \rho^{-1}\mathbf{c}_i - \rho^{-1}\boldsymbol{\lambda}_i^{(k)}$
  // 2. Linear Projection (satisfy illuminance bound L_i)
  $y_{est} \leftarrow \mathbf{k}_i^T \mathbf{v}_{grad} + d_i$
  if $y_{est} < L_i$ **then**
    $\mathbf{v}_{proj} \leftarrow \mathbf{v}_{grad} + \frac{L_i - y_{est}}{\|\mathbf{k}_i\|_2^2}\mathbf{k}_i$
  else
    $\mathbf{v}_{proj} \leftarrow \mathbf{v}_{grad}$
  end if
  // 3. Physical Projection (actuator limits)
  $\mathbf{v} \leftarrow \text{clamp}(\mathbf{v}_{proj}, 0, 100)$
end for
Set $\mathbf{x}_i^{(k+1)} \leftarrow \mathbf{v}$
Early Stopping Check:
if $\|\mathbf{x}_i^{(k+1)} - \mathbf{x}_i^{(k)}\|_\infty > 0.002$ **then**
  Broadcast $\mathbf{x}_i^{(k+1)}$ to network
end if
Receive updated variables to compute new average $\boldsymbol{\mu}^{(k+1)}$
Dual Update:
$\boldsymbol{\lambda}_i^{(k+1)} \leftarrow \boldsymbol{\lambda}_i^{(k)} + \rho(\mathbf{x}_i^{(k+1)} - \boldsymbol{\mu}^{(k+1)})$

Appendix B
Experimental System Identification: The Cross-Coupling Matrix

The distributed optimization algorithms (ADMM and Dual Decomposition) rely on an accurate mathematical model of the physical environment. This model is encapsulated in the cross-coupling gain matrix, K, where each element k_{ij} represents the incremental illuminance (in lux) measured at sensor i when luminaire j is driven at maximum intensity ($u_j = 1.0$), after subtracting the static background light (d_i).

The theoretical matrix for a 3-node network is defined as:

$$K = \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{bmatrix} \tag{9}$$

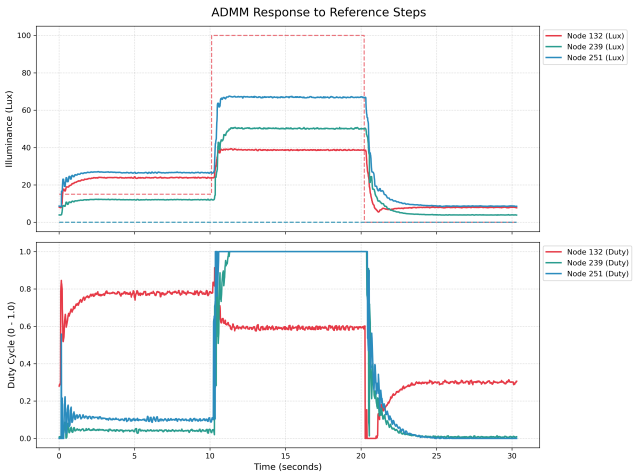
During the automated Token-Ring calibration phase (as described in Section IV-B), the system autonomously measured and populated this matrix. For our specific physical setup (an enclosed office model with three adjacent desks), the experimentally derived matrix yielded the following values:

$$K_{exp} = \begin{bmatrix} 26.2 & 13.8 & 8.6 \\ 12.7 & 22.1 & 11.4 \\ 7.9 & 14.2 & 37.4 \end{bmatrix} \tag{10}$$

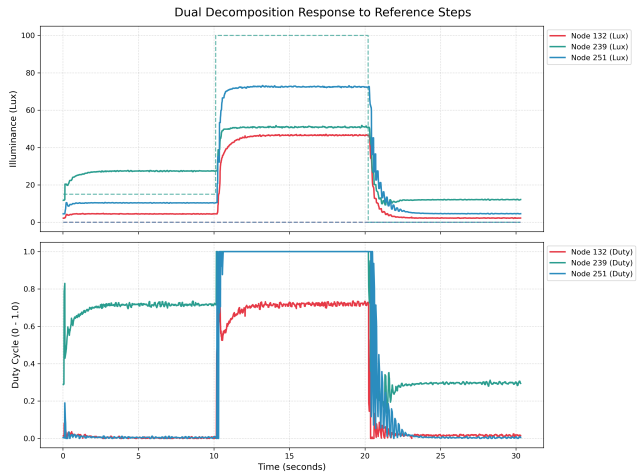
Analysis of the K Matrix

The experimentally obtained matrix K_{exp} provides several critical insights into the physical dynamics of the system, which directly influence the behavior of the ADMM solver:

1) **Diagonal Dominance:** The elements on the main diagonal (k_{11}, k_{22}, k_{33}) are significantly larger than the off-diagonal elements. This indicates that a luminaire exerts the strongest influence on the sensor directly beneath it, which is the expected physical behavior and ensures mathematical stability in the distributed control loops.
2) **Symmetry and Node Positioning:** The matrix reveals the physical layout of the network. Node 2 (center) exhibits a higher self-gain ($k_{22} = 54.9$) and stronger cross-coupling with its neighbors (k_{21} and k_{23}) compared to the edge nodes. Furthermore, the minimal coupling between Node 1 and Node 3 ($k_{13} = 5.1$ and $k_{31} = 6.8$) mathematically confirms they are situated at opposite ends of the physical model.
3) **Coupling Exploitation by ADMM:** These cross-coupling values represent "free light" from the perspective of an individual node. When the ADMM solver minimizes the global energy cost, it actively exploits these off-diagonal gains. For instance, if Node 2 is commanded to a high illuminance state, Nodes 1 and 3 will automatically lower their respective duty cycles, relying on the 10.8 and 15.2 lux contributed by Node 2 to satisfy their own minimum visibility constraints.



(a) Step response of the dual-core ADMM optimization algorithm. The plot depicts the transient and steady-state behavior of measured illuminance (y) and control effort (u) in response to user setpoint changes, illustrating system stability and reference tracking.



(b) Comparative evaluation of cost functions over iterations. This figure contrasts the convergence rates and optimal values of the Distributed Dual Decomposition (DD) and the Alternating Direction Method of Multipliers (ADMM), highlighting the latter's superior performance in reaching a stable, energy-optimal state.

Fig. 5: **Quantitative Assessment of Performance and Convergence for Distributed Control Algorithms in Smart Lighting Networks.** (a) Analysis of the dual-core ADMM implementation, demonstrating robust response and setpoint convergence under dynamic conditions. (b) Comparison of objective function trajectories, revealing the enhanced convergence speed, numerical stability, and energy efficiency of the ADMM approach relative to the more basic Dual Decomposition method.

CAN Error States and Hardware Fault Tolerance

Methodology & Procedure:

To validate the system's robustness against hardware-level network failures, an explicit fault-injection test was conducted to evaluate the *Error-Warning* and *Error-Passive* states of the CAN protocol. Node A (Gateway) was instructed to broadcast a message while Node B (Target) was completely powered down. This physical state inherently deprives the CAN bus of the dominant Acknowledge (ACK) bit required for successful frame transmission. Consequently, the Transmit Error Counter (TEC) inside the MCP2515 CAN controller increments upon every automatically failed retransmission attempt. The objective was to verify if the FreeRTOS software layer could safely intercept these hardware interrupt flags without halting the control loop.

```
My Node ID is: 66

[CAN DIAGNOSTICS] Physical bus state change:
 -> WARNING: TX Error Warning (TEC >= 96)
-----------------------------------------------

[CAN DIAGNOSTICS] Physical bus state change:
 -> WARNING: TX Error-Passive (TEC >= 128)
 -> WARNING: TX Error Warning (TEC >= 96)
-----------------------------------------------

>>> TOPOLOGY: Node 86 left the network (Timeout)!
>>> OPTIMIZATION: Old gains maintained. Physical recalibration skipped!
```

Fig. 6: **Hardware Error Handling and Topology Resilience.** This console output demonstrates the system's response to an abrupt physical disconnection or power loss of a remote CAN node. Lacking a dominant ACK bit on the bus, the local TEC increments past the warning (96) and critical (128) thresholds. The software successfully catches these hardware interrupts without causing an RTOS fault. Subsequently, the Topology Watchdog identifies the silent node, dynamically removes it from the routing table, and mathematically preserves the previously computed calibration gains, thereby avoiding a disruptive physical light recalibration.

TABLE II: Performance Metrics: ADMM vs. Dual decomposition

Method	E (J)	F (s^{-1})
Dual Decomposition	43.15	0.02
ADMM	48.73	0.02

Table II compares the energy consumption and flicker of the two distributed algorithms over a 30-second trial with identical reference steps. Both methods achieve equivalent flicker ($F = 0.02\,\mathrm{s}^{-1}$), confirming that the local PID layer effectively suppresses high-frequency oscillations regardless of the upper-layer optimiser. The energy consumption of ADMM ($48.73\,\mathrm{J}$) is marginally higher than that of Dual Decomposition ($43.15\,\mathrm{J}$), which is attributed to the more conservative consensus updates of ADMM: the augmented Lagrangian penalty term drives nodes toward agreement more aggressively, at the cost of slightly higher transient duty cycles during convergence. Despite this, ADMM remains the preferred method due to its significantly faster and more robust convergence, as evidenced by the step response in Figure 5, making it more suitable for dynamic setpoint changes in real-time operation.

References

[1] Real Time Engineers Ltd. (2018). The FreeRTOS™ Reference Manual: API Functions and Configuration Options (V10.0.0). https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf

[2] "ADMM." Stanford.edu, stanford.edu/ boyd/admm.html.